Linguaggi di Programmazione

Analisi lessicale: espressioni regolari, DFA, NFA

Roberto Gorrieri

Alma Mater Studiorum - Università di Bologna

Analisi lessicale

Analisi Lessicale

Analisi Lessicale:
Riconoscere nella stringa in ingresso gruppi/sequenze di simboli che corrispondono a specifiche categorie sintattiche (ad esempio identificatori, parole riservate, operatori, parentesi, ...)



La stringa in input è trasformata in una sequenza di simboli astratti, detti **token**.

Cos'è un token?

Un token è una coppia $< nome, valore >$ (e.g., $< Ide, x1 >$), dove:

- *nome*: simbolo astratto che rappresenta una categoria sintattica (e.g., *Ide* per identificatore)
- *valore*: una sequenza di simboli del testo in ingresso (e.g., $x1$)

Dato il token $< Ide, x1 >$, diciamo che:

- **nome** (*Ide*) è l'informazione che identifica la classe di token (identificatori);
- **valore** ($x1$) è l'informazione che identifica uno specifico token (l'identificatore $x1$);
- **pattern** è la descrizione generale della forma dei valori di una classe di token, ad esempio $(x|y)(x|y|0|1)^*$ (una espressione regolare);
- **lessema** è una stringa istanza di un pattern. Ad esempio, $x1$ è istanza del pattern $(x|y)(x|y|0|1)^*$.

Ad ogni nome di categoria sintattica è associato un pattern che specifica i possibili valori che possono essere presi per quel nome, come lessemi.

Esempio

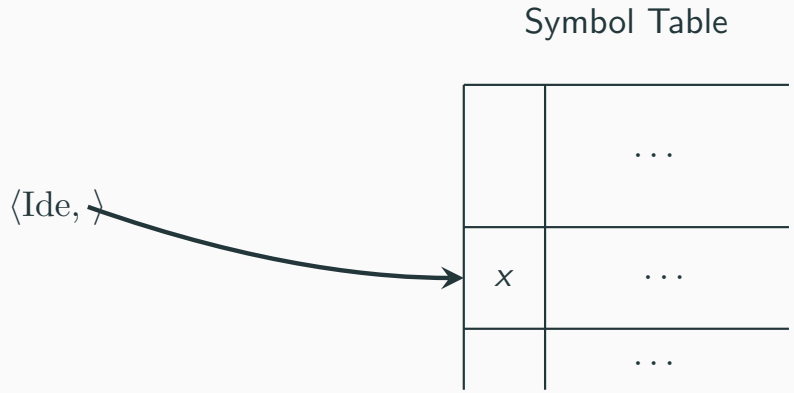
Dalla stringa scritta nel linguaggio C

$$if\ (x == 0)\ printf(\text{"}zero\text{"})$$

un analizzatore lessicale potrebbe produrre la seguente sequenza di token:

$$< IF > \quad < (> \quad < Ide, x > \quad < OPREL, ==> \quad < CONST\text{-}NUM, 0 > \quad <) >$$
$$< Ide, printf > \quad < (> \quad < CONST\text{-}STRING, zero > \quad <) >$$

Come vedete, non tutti i token sono coppie... Normalmente lo scanner associa agli identificatori un indirizzo nella tabella dei simboli: $< Ide, x >$ è in realtà $< Ide$, puntatore alla tabella dei simboli$>$.



Espressioni regolari

Sintassi delle spressioni regolari

Fissato un alfabeto $A = \{a_1, a_2, ..., a_n\}$, definiamo le espressioni regolari su A con la seguente BNF:

$$r \ ::= \ \emptyset \mid \epsilon \mid a \mid r \cdot r \mid r|r \mid r^*$$

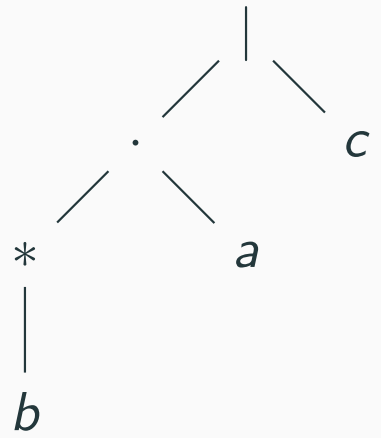
dove a è un simbolo dell'alfabeto A. L'operatore $\cdot$ è detto concatenazione, l'operatore $|$ è detto disgiunzione, e l'operatore $*$ ripetizione o stella di Kleene.
Questa è una sintassi astratta, quindi ambigua. Per disambiguarla, occorre fissare le associatività degli operatori e le priorità fra gli operatori. Si assume che:

- concatenazione, disgiunzione, ripetizione associno a sinistra;
- la precedenza tra operatori sia $* > \cdot > |$.

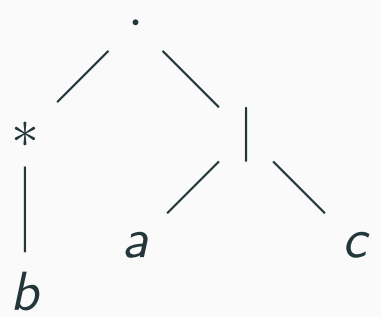
Si può usare zucchero sintattico (parentesi) per disambiguare. Inoltre, per semplicità, la concatenazione $\cdot$ è di solito omessa.

Sintassi delle spressioni regolari (2)

In base alle assunzioni fatte, l'espressione regolare $b^*a|c$ corrisponde all'albero sintattico



mentre se volgiamo quest'altro albero sintattico astratto, dovremo scrivere $b^*(a|c)$.



Linguaggio denotato da un'espressione regolare

Dato un alfabeto A, definiamo la funzione $\mathcal{L} : \text{Exp-Reg} \to \mathcal{P}(A^*)$, che definisce la semantica delle espressioni regolari, come segue:

$$\mathcal{L}[\emptyset] = \emptyset$$
$$\mathcal{L}[\epsilon] = \{\epsilon\}$$
$$\mathcal{L}[a] = \{a\}$$
$$\mathcal{L}[r_1 \cdot r_2] = \mathcal{L}[r_1] \cdot \mathcal{L}[r_2]$$
$$\mathcal{L}[r_1 | r_2] = \mathcal{L}[r_1] \cup \mathcal{L}[r_2]$$
$$\mathcal{L}[r^*] = (\mathcal{L}[r])^*$$

Se usate le parentesi: $\mathcal{L}[(r)] = \mathcal{L}[r]$.

N.B. Operatori sintattici corrispondono ad operatori semantici.

Ricorda che:

- $L_1 \cdot L_2 = \{w_1 w_2 | w_1 \in L_1 \wedge w_2 \in L_2\}$
- $L_1 \cup L_2 = \{w | w \in L_1 \vee w \in L_2\}$
- $L^0 = \{\epsilon\}$, $L^{n+1} = L \cdot L^n$, $L^* = \bigcup_{n \geq 0} L^n$

Linguaggio regolare

Linguaggio regolare

Un linguaggio $L \subseteq A^*$ è detto regolare se e solo se esiste una espressione regolare r tale che $L = \mathcal{L}[r]$.

Proposizione Ogni linguaggio finito è regolare.

Ad esempio $L = \{a, bc\}$ è denotato dall'espressione regolare $r = a|bc$.
$\mathcal{L}[a|bc] = \mathcal{L}[a] \cup \mathcal{L}[bc] = \{a\} \cup \mathcal{L}[b] \cdot \mathcal{L}[c] = \{a\} \cup \{b\} \cdot \{c\} = \{a, bc\}$.
N.B. Non serve l'operatore $*$ per rappresentare linguaggi finiti.

Ma esistono anche linguaggi regolari infiniti. Ad esempio:

- $\mathcal{L}[a^*b] = \mathcal{L}[a^*] \cdot \mathcal{L}[b] = (\mathcal{L}[a])^* \cdot \{b\} = \{a\}^* \cdot \{b\} = \bigcup_{n \geq 0} \{a\}^n \cdot \{b\} = \{\epsilon, a, aa, aaa, \ldots\} \cdot \{b\} = \{a^n b \mid n \geq 0\}$.
- $\mathcal{L}[(a|b)b^*] = \mathcal{L}[a|b] \cdot \mathcal{L}[b^*] = (\mathcal{L}[a] \cup \mathcal{L}[b]) \cdot (\mathcal{L}[b])^* = \{a, b\} \cdot \{b\}^* = \{ab^n \mid n \geq 0\} \cup \{b^n \mid n \geq 1\}$.

Esempi

Prendiamo $A = \{0, 1\}$. Consideriamo le seguenti espressioni regolari e descriviamo il linguaggio che rappresentano.

- 0^*10^*
  - $L_1 = \{w \in A^* \mid w \text{ contiene un solo } 1\}$
- $(0|1)^*1(0|1)^*$
  - $L_2 = \{w \in A^* \mid w \text{ contiene almeno un } 1\}$
- $(0|1)^*001(0|1)^*$
  - $L_3 = \{w \in A^* \mid w \text{ contiene } 001 \text{ come sottostringa}\}$
- $1^*(011^*)^*$
  - $L_4 = \{w \in A^* \mid \text{in } w \text{ ogni occorrenza di } 0$ è seguita immediatamente da almeno un $1\}$
- $(0|1)^*(0|1)^*$
  - $L_5 = \{w \in A^* \mid w \text{ è di lunghezza pari}\}$
- $(0|1)^*1$
  - $L_6 = \{w \in A^* \mid w \text{ termina con } 1\}$

Operatori ausiliari

- Ripetizione positiva r^+: è lo stesso che dire rr^* oppure r^*r
- Possibilità $r?$: ovvero $(r|\epsilon)$
- Elenco $[a_1, ..., a_n]$: $a_1|a_2|...|a_n$ per $a_i \in A$
  - se gli a_i sono ordinati, in modo che $a_i < a_{i+1}$, allora $[a_1 - a_n]$ sta per $a_1|a_2|...|a_n$.

Ad esempio, i numeri decimali senza segno hanno come alfabeto $\{0, 1, ..., 9, .\}$ e sono definiti come

$$[0 - 9]^+(\epsilon|. [0 - 9]^+)$$

O alternativamente

$$[0 - 9]^+(. [0 - 9]^+)?$$

Definizioni regolari

Una definizione regolare su alfabeto A è costituita da una lista di definizioni

$$d_1 := r_1$$
$$d_2 := r_2$$
$$\ldots$$
$$d_k := r_k$$

dove i vari d_i sono simboli "nuovi" e ogni r_i è una espressione regolare sull'alfabeto esteso $A \cup \{d_1, \ldots, d_k\}$. Ad esempio, i numeri decimali con segno sono definiti come:

$$\begin{aligned} \mathit{numconsegno} &:= \mathit{segno}\ \mathit{cifre}\ (.\mathit{cifre})? \\ \mathit{segno} &:= + \mid - \\ \mathit{cifre} &:= \mathit{cifra}^+ \\ \mathit{cifra} &:= [0-9] \end{aligned}$$

Esempio: identificatori e password

Gli identificatori sono definiti in modo diverso in diversi linguaggi di programmazione. Una definizione regolare semplice è la seguente:

$$\begin{aligned} \mathit{identifier} &:= \mathit{letter}\ (\mathit{letter} \mid \mathit{digit})^* \\ \mathit{letter} &:= [a - zA - Z] \\ \mathit{digit} &:= [0 - 9] \end{aligned}$$

ovvero un identificatore è una qualunque sequenza alfanumerica che inizia con una lettera.

Esercizio: Fornire una definizione regolare per un tipo di password alfanumerica che deve soddisfare i seguenti vincoli:

- iniziare con una lettera maiuscola;
- contenere almeno due cifre non contigue;
- terminare con una lettera minuscola.

Equivalenza tra espressioni regolari

Due espressioni regolari r e s sono equivalenti se e solo se $\mathcal{L}[r] = \mathcal{L}[s]$ (cioè denotano lo stesso linguaggio) e lo denotiamo con $r \simeq s$.
Esistono molte leggi per $\simeq$, alcune sono le seguenti:

$r\|s \simeq s\|r$	$\|$ è commutativa
$r\|(s\|t) \simeq (r\|s)\|t$	$\|$ è associativa
$r\|r \simeq r$	$\|$ è idemptotente
$r \cdot (s \cdot t) \simeq (r \cdot s) \cdot t$	$\cdot$ è associativa
$\epsilon \cdot r \simeq r \cdot \epsilon$	ϵ è elemento neutro per $\cdot$
$(r^*)^* \simeq r^*$	$*$ è idemptotente
$r \cdot (s\|t) \simeq (r \cdot s)\|(r \cdot t)$	$\cdot$ distribuisce a sx su $\|$
$(r\|s) \cdot t \simeq (r \cdot t)\|(s \cdot t)$	$\cdot$ distribuisce a dx su $\|$
$\emptyset^* \simeq \epsilon$	$r\|\emptyset \simeq r$
$r \cdot \emptyset \simeq \emptyset \simeq \emptyset \cdot r$	$(\epsilon\|r)^* \simeq r^*$
$(r^* s^*)^* \simeq (r\|s)^*$	

Equivalenza tra espressioni regolari (2)

Dimostrare queste leggi non è sempre facile, ma in alcuni casi lo è.

- $\mathcal{L}[r|s] = \mathcal{L}[r] \cup \mathcal{L}[s] = \mathcal{L}[s] \cup \mathcal{L}[r] = \mathcal{L}[s|r]$
- $\mathcal{L}[r|r] = \mathcal{L}[r] \cup \mathcal{L}[r] = \mathcal{L}[r]$
- $\mathcal{L}[r \cdot \epsilon] = \mathcal{L}[r] \cdot \mathcal{L}[\epsilon] = \mathcal{L}[r] \cdot \{\epsilon\} = \mathcal{L}[r]$
- $\mathcal{L}[\emptyset^*] = (\mathcal{L}[\emptyset])^* = \emptyset^* = \emptyset^0 \cup \emptyset^1 \cup \emptyset^2 \cup \ldots = \{\epsilon\} \cup \emptyset \cup \emptyset \cup \ldots = \{\epsilon\} = \mathcal{L}[\epsilon]$
- $\mathcal{L}[r \cdot \emptyset] = \mathcal{L}[r] \cdot \mathcal{L}[\emptyset] = \mathcal{L}[r] \cdot \emptyset = \emptyset = \mathcal{L}[\emptyset]$

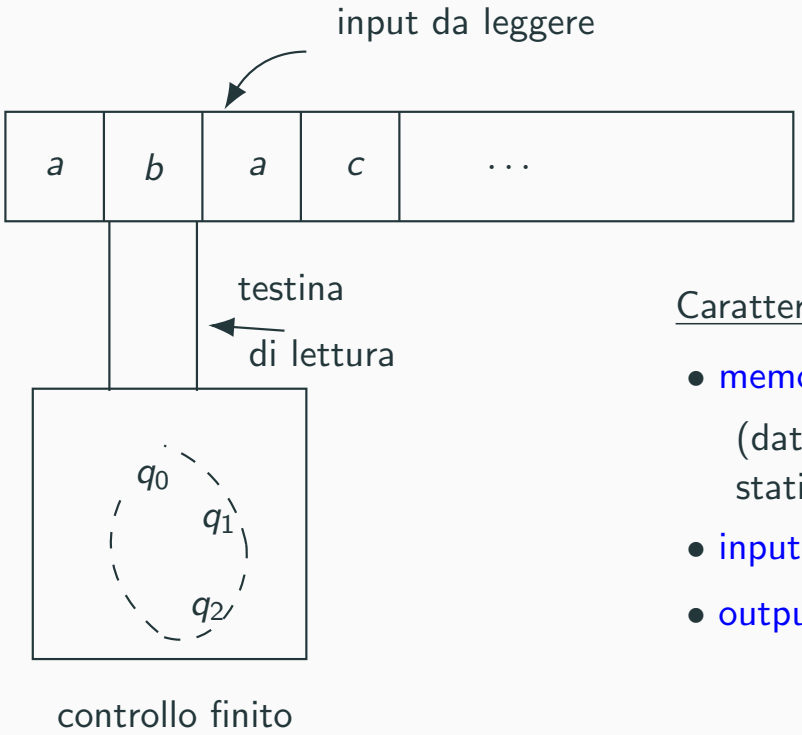
Automi finiti: DFA e NFA

Automi finiti come riconoscitori di linguaggi regolari

Le espressioni regolari servono per specificare il pattern di una categoria sintattica, ovvero la forma dei possibili lessemi.
Ma come **riconoscere** se una certa sequenza in ingresso è un lessema per una certa categoria sintattica?

Automi (a stati) finiti

che possono essere deterministici (DFA) o nondeterministici (NFA).



Caratteristiche:

- memoria finita
  (data dal numero degli stati: $q_0, \ldots, q_n$)
- input: stringa da riconoscere
- output: 1 bit (Si/No)

Automi (a stati) finiti – funzionamento

Descrizione iniziale:

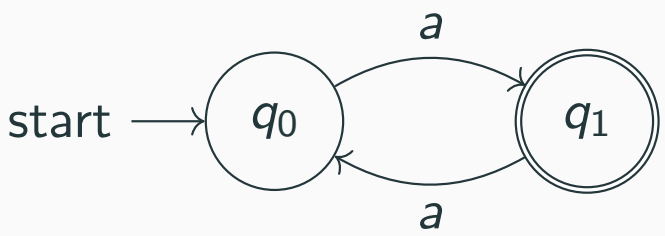
- testina di lettura posizionata sul primo carattere dell'input
- controllo posizionato sullo stato iniziale q_0

Funzionamento:

- Ripete
  - legge il carattere in input e, in base allo stato in cui si trova, decide:
    - di cambiare stato (eventualmente, può rimane sullo stesso stato)
    - e di spostare la testina sull'input successivo
- fino a che
  - ha finito di leggere l'input (e riconosce la stringa se ha raggiunto uno stato finale),
  - oppure si è bloccato prima perché per la coppia (stato corrente, input attuale) non era specificato uno stato successivo.

Diagrammi di transizione

Il funzionamento di un automa finito è ben rappresentato attraverso un diagramma di transizione.



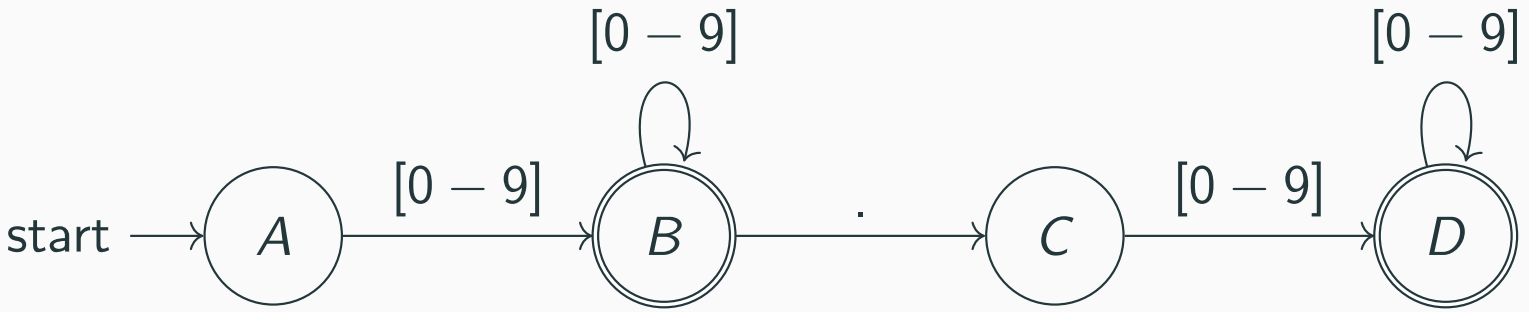
In questo caso, q_0 è lo stato iniziale e q_1 è lo stato finale.
Riconoscere una stringa w significa trovare un cammino etichettato w sul grafo a partire dallo stato iniziale che finisce su uno stato finale. Per il diagramma qui rappresentato, il linguaggio riconosciuto è:

$$L = \{a^{2n+1} | n \geq 0\} = \{a^n | n \text{ è dispari } \} = \mathcal{L}[a(aa)^*]$$

N.B.: se in input ho $w = ab$, l'automa si blocca in q_1 senza che la lettura dell'input sia completata.

Esempio

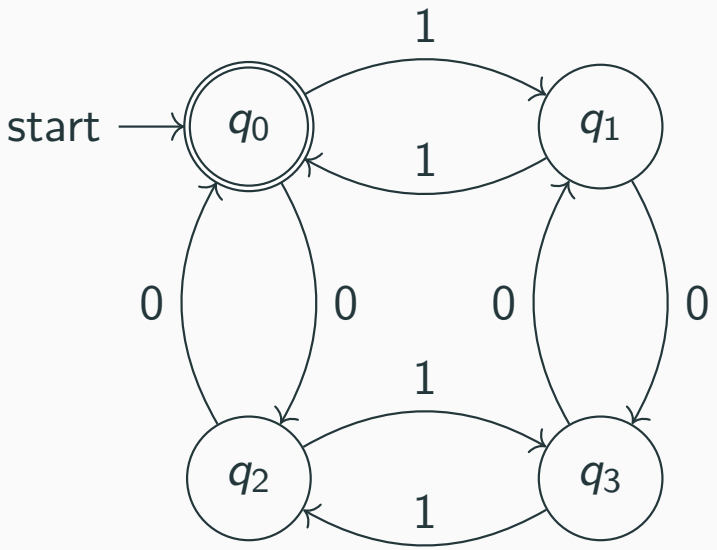
Diagramma di transizione corrispondente all'espressione regolare (estesa) $[0-9]^{+}(\epsilon|.\,[0-9]^{+})$ che descrive i numeri decimali senza segno:



In questo caso, l'arco $[0-9]$ riassume 10 diverse transizioni, una per ciascun simbolo $0, \ldots, 9$. Questo automa ci permette di riconoscere se l'input è un numero decimale senza segno.
(Vedremo più avanti una costruzione canonica per mappare espressioni regolari su automi finiti.)

Esempio (1)

$L[M_1] = \{w \in \{0,1\}^* \mid \text{in } w \text{ il numero di 0 e 1 è pari}\}$

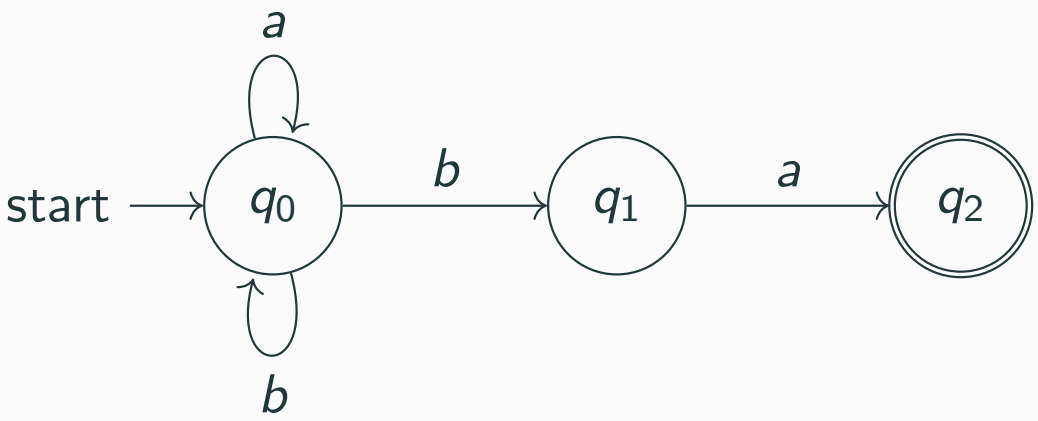


N.B.: automa deterministico: per ogni coppia (q, s), con $q \in \{q_0, q_1, q_2, q_3\}$ e $s \in \{0, 1\}$, esiste esattamente una sola possibile mossa.

Esercizio: Costruire l'automa M che riconosce il linguaggio $L[M] = \{w \in \{0,1\}^* \mid \text{in } w \text{ il numero di 0 e 1 è dispari}\}$.

Esempio (2)

$L[M_2] = \mathcal{L}[(a|b)^* ba]$

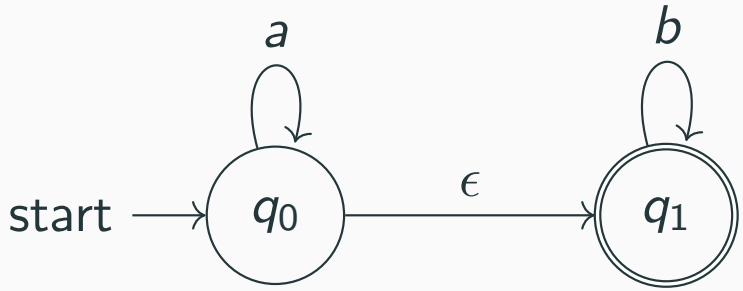


- automa nondeterministico:
  - (q_0, b) offre due mosse alternative (o su q_0 o su q_1);
  - (q_1, b) non offre mosse;
  - $(q_2, a/b)$ non offre mosse.

N.B.: $ba \in L[M_2]$ perché esiste un cammino da q_0 a q_2 etichettato ba.
(Ma esiste anche un altro cammino per ba che da q_0 porta su q_0.)

Esempio (3)

$L[M_3] = \mathcal{L}[a^* b^*]$

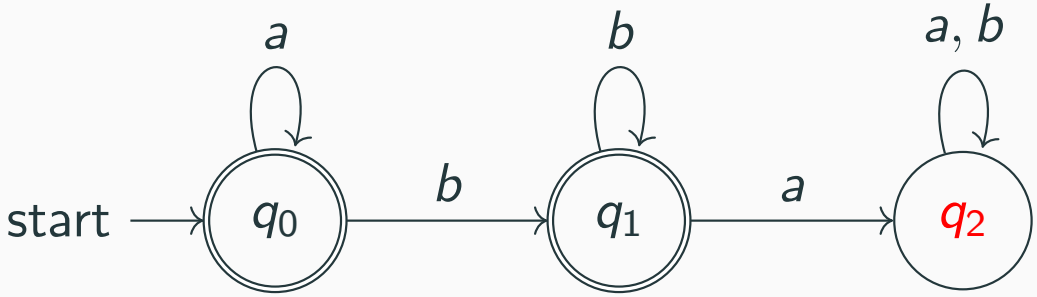


- automa nondeterministico:
  - (q_0, b) non offre mosse;
  - (q_1, a) non offre mosse;
  - (q_0, ϵ, q_1) rappresenta una mossa "silente" in cui non viene letto l'input. Fonte di nondeterminismo, perché, se l'input da leggere è a, l'automa dallo stato q_0 può scegliere di leggere l'input a (senza cambiare stato), o di non leggere l'input (cambiando stato): due mosse alternative!

N.B.: $\epsilon \in L[M_3]$ perché esiste un cammino da q_0 a q_1 etichettato ϵ.

Esempio (4)

Se vogliamo un automa deterministico per $\mathcal{L}[a^*b^*]$ possiamo definire:



dove q_2 è uno stato "pozzo" di errore. L'automa è deterministico perché

- non ci sono transizioni etichettate ϵ;
- da ogni stato, per ognuno dei due simboli a e b, esce una ed una sola transizione.

N.B.: ϵ è riconosciuta perché esiste un cammino che va dallo stato iniziale q_0 allo stato finale q_0 (cammino vuoto) senza leggere nessun simbolo in input.

Automi Finiti Nondeterministici (NFA)

NFA
Un automa finito nondeterministico (NFA) è una quintupla $(\Sigma, Q, \delta, q_o, F)$ dove:

- Σ è un alfabeto finito di simboli in input;
- Q è un insieme finito di stati;
- $q_o \in Q$ è lo stato iniziale;
- $F \subseteq Q$ è l'insieme di stati finali;
- δ è la funzione di transizione con tipo $\delta : Q \times (\Sigma \cup \{\epsilon\}) \to \mathcal{P}(Q)$, ovvero $\delta(q, \sigma) = Q' \subseteq Q$.

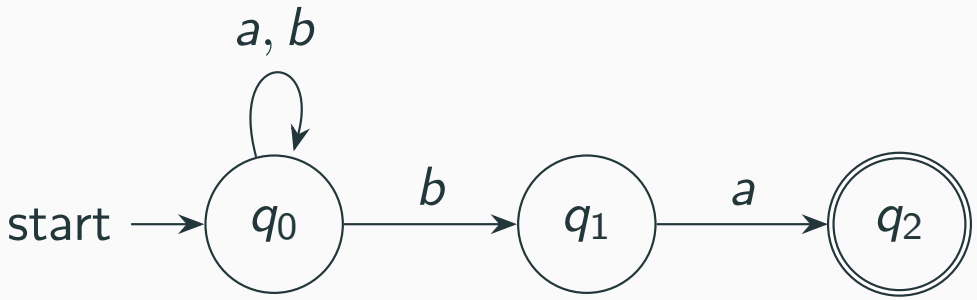
N.B.: Se $\delta(q, \sigma) = \emptyset$, allora da q non parte nessuna transizione etichettata σ.

Esempio

Diamo la definizione formale di un NFA $N = (\Sigma, Q, \delta, q_o, F)$:
$\Sigma = \{a, b\}$ $Q = \{q_0, q_1, q_2\}$ q_0 è lo stato iniziale $F = \{q_2\}$
Rappresentazione della funzione di transizione δ in forma tabellare:

δ	a	b	ϵ
q_0	$\{q_0\}$	$\{q_0, q_1\}$	$\emptyset$
q_1	$\{q_2\}$	$\emptyset$	$\emptyset$
q_2	$\emptyset$	$\emptyset$	$\emptyset$

Come rappresentare questo NFA N come diagramma di transizione?

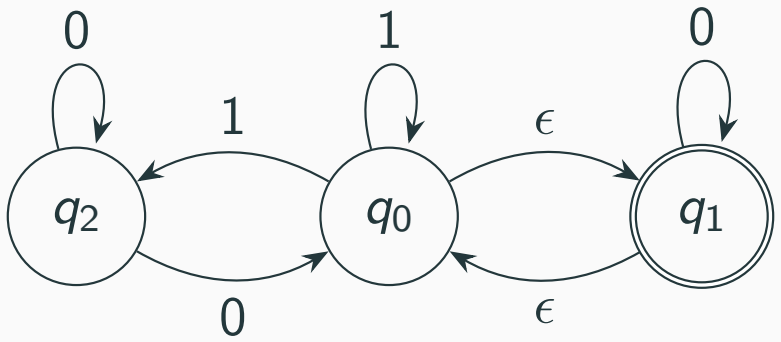


Esercizio, con soluzione

Disegnate il diagramma di transizione per $M = (\Sigma, Q, \delta, F)$ dove
$\Sigma = \{0, 1\}$ $Q = \{q_0, q_1, q_2\}$ q_0 è lo stato iniziale $F = \{q_1\}$
e la funzione di transizione δ è definita dalla seguente tabella:

δ	0	1	ϵ
q_0	$\emptyset$	$\{q_0, q_2\}$	$\{q_1\}$
q_1	$\{q_1\}$	$\emptyset$	$\{q_0\}$
q_2	$\{q_0, q_2\}$	$\emptyset$	$\emptyset$

Ecco la soluzione:



Linguaggio riconosciuto/accettato

Definizione informale: Un NFA $N = (\Sigma, Q, \delta, q_o, F)$ accetta $w = a_1...a_n$ se e solo se nel diagramma di transizione esiste un cammino da q_0 ad uno stato in F nel quale la stringa che si ottiene concatenando le etichette degli archi percorsi è esattamente w.

Ad esempio, considerando l'automa della slide precedente, la stringa 10 è riconosciuta perché esiste un cammino dallo stato iniziale q_0 allo stato finale q_1 che "legge" 10:

$$q_0 \xrightarrow{1} q_0 \xrightarrow{\epsilon} q_1 \xrightarrow{0} q_1$$

Più formalmente, indichiamo la descrizione istantanea come una coppia (q, w), dove q è lo stato corrente e w l'input da leggere, e definiamo:

Linguaggio riconosciuto/accettato - formalmente

Mossa (con $\sigma \in \Sigma \cup \{\epsilon\}$ e $w \in \Sigma^*$):

$$\frac{q' \in \delta(q, \sigma)}{(q, \sigma w) \vdash_N (q', w)}$$

Cammino (chiusura riflessiva e transitiva di $\vdash_N$):

$$\frac{}{(q, w) \vdash_N^* (q, w)} \qquad \frac{(q, w) \vdash_N^* (q', w') \quad (q', w') \vdash_N (q'', w'')}{(q, w) \vdash_N^* (q'', w'')}$$

Accettazione/Riconoscimento:

$$w \in L[N] \iff \exists q \in F : (q_0, w) \vdash_N^* (q, \epsilon).$$

Il linguaggio accettato da N, indicato con $L[N]$ è

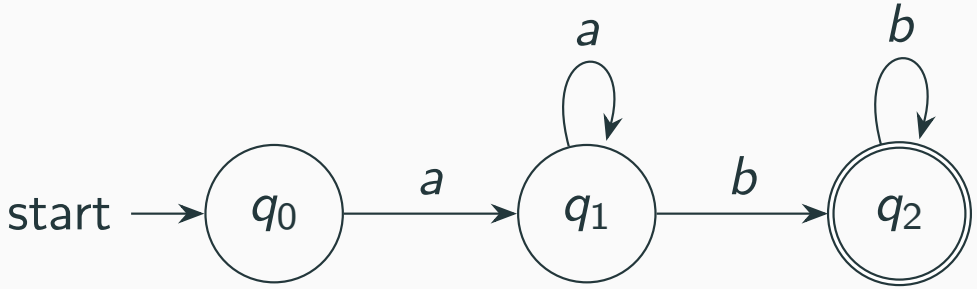
$$\boxed{L[N] = \{w \in \Sigma^* | \exists q \in F : (q_0, w) \vdash_N^* (q, \epsilon)\}}$$

Due NFA N_1, N_2 si dicono **equivalenti** se e solo se accettano lo stesso linguaggio, cioè se $L[N_1] = L[N_2]$.

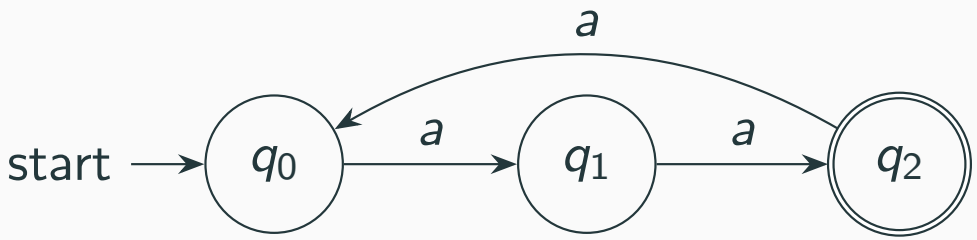
Esercizi, con soluzione

Costruire un NFA (o un DFA) per i seguenti linguaggi:

$L_1 = \{a^n b^m \mid n, m \geq 1\}$

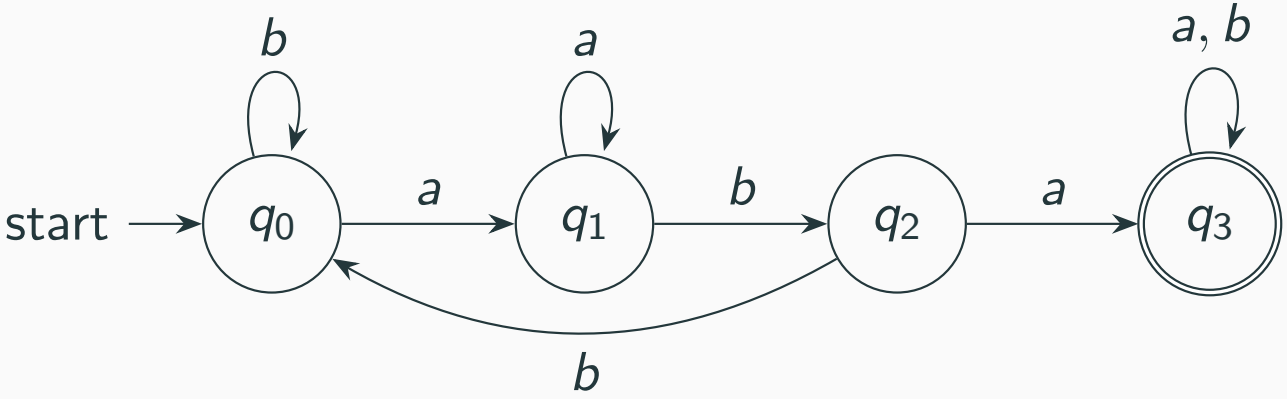


$L_2 = \{a^{3k+2} \mid k \geq 0\}$

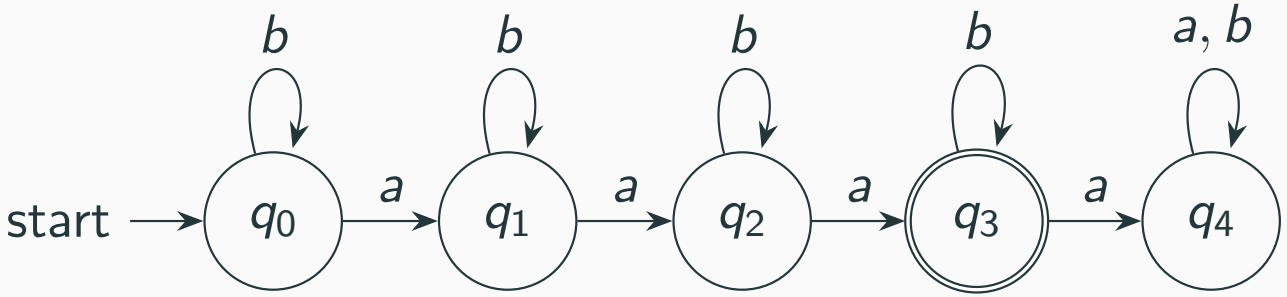


Esercizi, con soluzione (2)

$L_3 = \{w \in \{a, b\}^* \mid w \text{ contiene la sottostringa } aba\}$



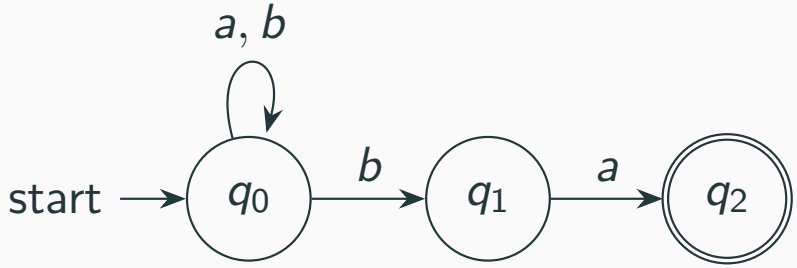
$L_4 = \{w \in \{a, b\}^* \mid w \text{ contiene esattamente tre } a\}$



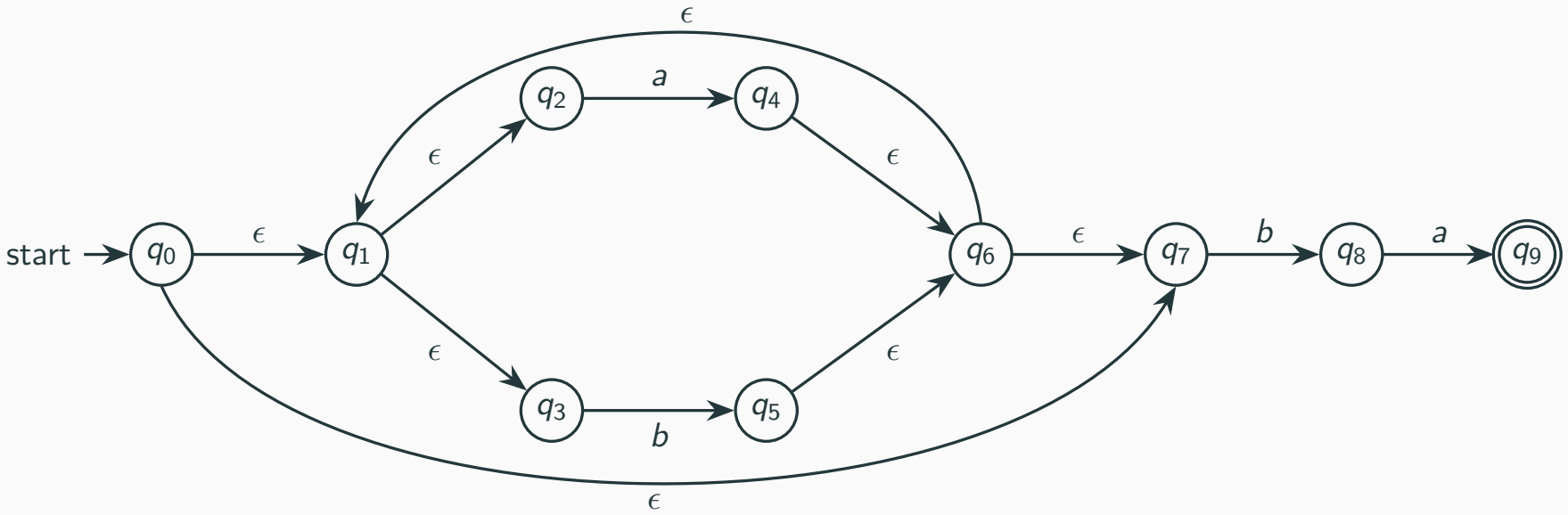
Domanda: Dato un NFA N, quanto altri NFA sono ad esso equivalenti?

Esempio di NFA equivalenti

$L[N_1] = \mathcal{L}[(a|b)^* ba]$



$L[N_2] = \mathcal{L}[(a|b)^* ba]$



NFA e DFA

Gli NFA (Nondeterministic Finite Automata) sono

- "comodi": facile costruirli, ma
- inefficienti: accettare w significa cercare un cammino su un grafo "nondeterministico" $\Rightarrow$ tante potenziali strade alternative (possibili failure $\Rightarrow$ backtracking)

DFA: Deterministic Finite Automata

- $\delta(q, \sigma)$ è sempre un singoletto (una sola mossa possibile);
- non ci sono le mosse ϵ.
  Quindi:
- garantita la scansione completa dell'input (non si blocca mai);
- in un tempo $O(n)$ - dove n è la lunghezza di w - sappiamo se w è accettata o meno;
- ma più difficili da definire.

Automi Finiti Deterministici (DFA)

DFA

Un automa finito deterministico (DFA) è una quintupla $(\Sigma, Q, \delta, q_o, F)$ dove Σ, Q, q_o, F sono definiti come per un NFA, mentre la funzione di transizione δ ha tipo $\delta : Q \times \Sigma \to Q$, cioè $\delta(q, \sigma) = q'$.

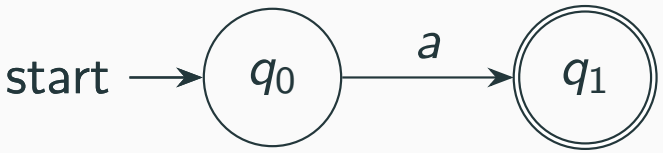
Un DFA è un particolare tipo di NFA tale che:

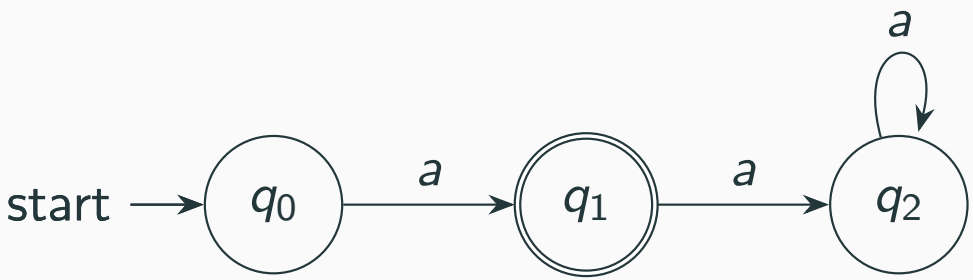
- non ci sono transizioni ϵ, cioè $\forall q \in Q, \delta(q, \epsilon) = \emptyset$;
- l'insieme delle mosse possibili è sempre un singoletto, cioè $\forall \sigma \in \Sigma, \forall q \in Q, \exists q' \in Q : \delta(q, \sigma) = \{q'\}$.

Ed infatti, $DFA \subset NFA$.

NFA o DFA?

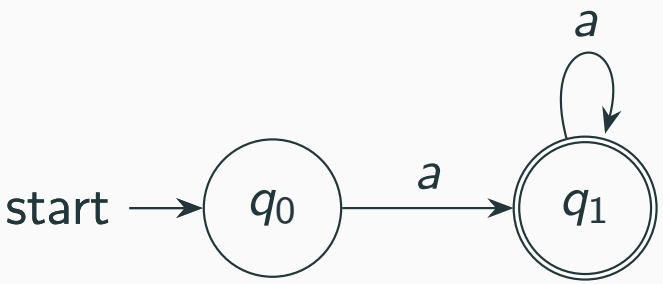
Se $\Sigma = \{a\}$, allora i seguenti automi sono DFA o NFA?

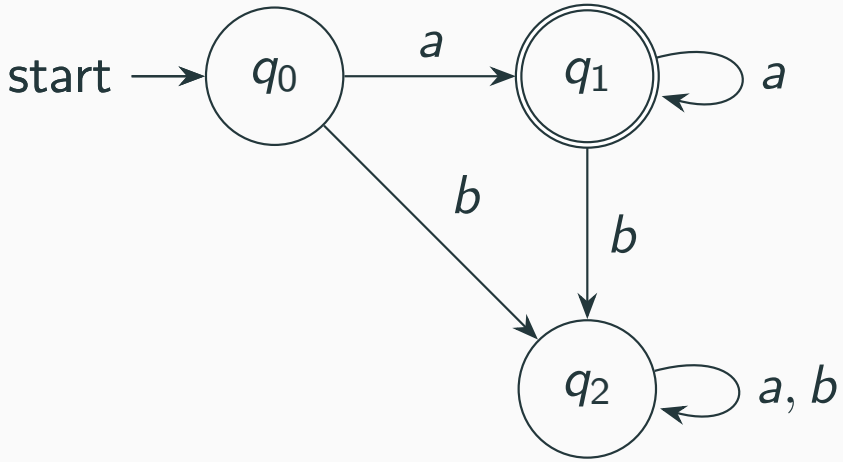




NFA o DFA? (2)

Se $\Sigma = \{a, b\}$, allora i seguenti automi sono DFA o NFA?





DFA e NFA sono equivalenti

Espressività di NFA e DFA

Vogliamo dimostrare che i DFA sono tanto espressivi quanto gli NFA, sebbene siano un sottoinsieme proprio degli NFA.

Per ogni NFA, è possibile costruire un DFA ad esso equivalente!

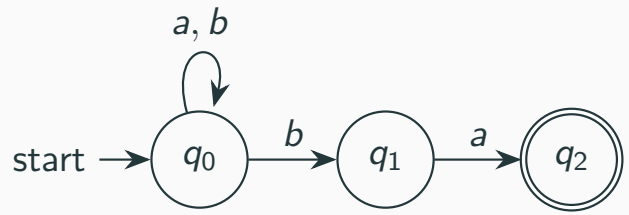
Come fare?

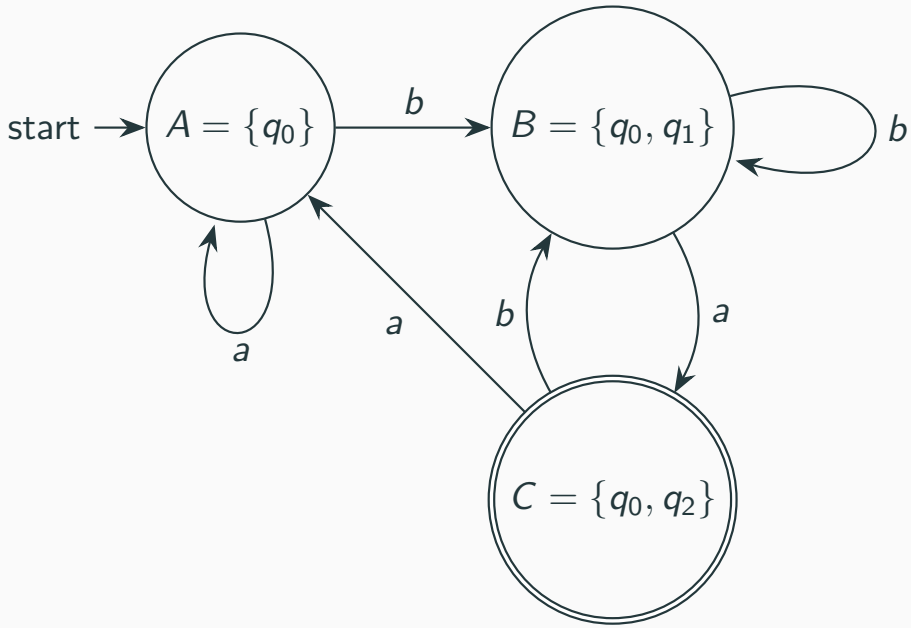
- L'idea è "seguire" contemporaneamente tutti i cammini alternativi dell'NFA,
- quindi gli stati del DFA che andiamo a costruire sono costituiti da insiemi di stati dell'NFA.

Si usa la tecnica denominata "costruzione per sottoinsiemi" (subset construction).

Primo esempio semplice

NFA



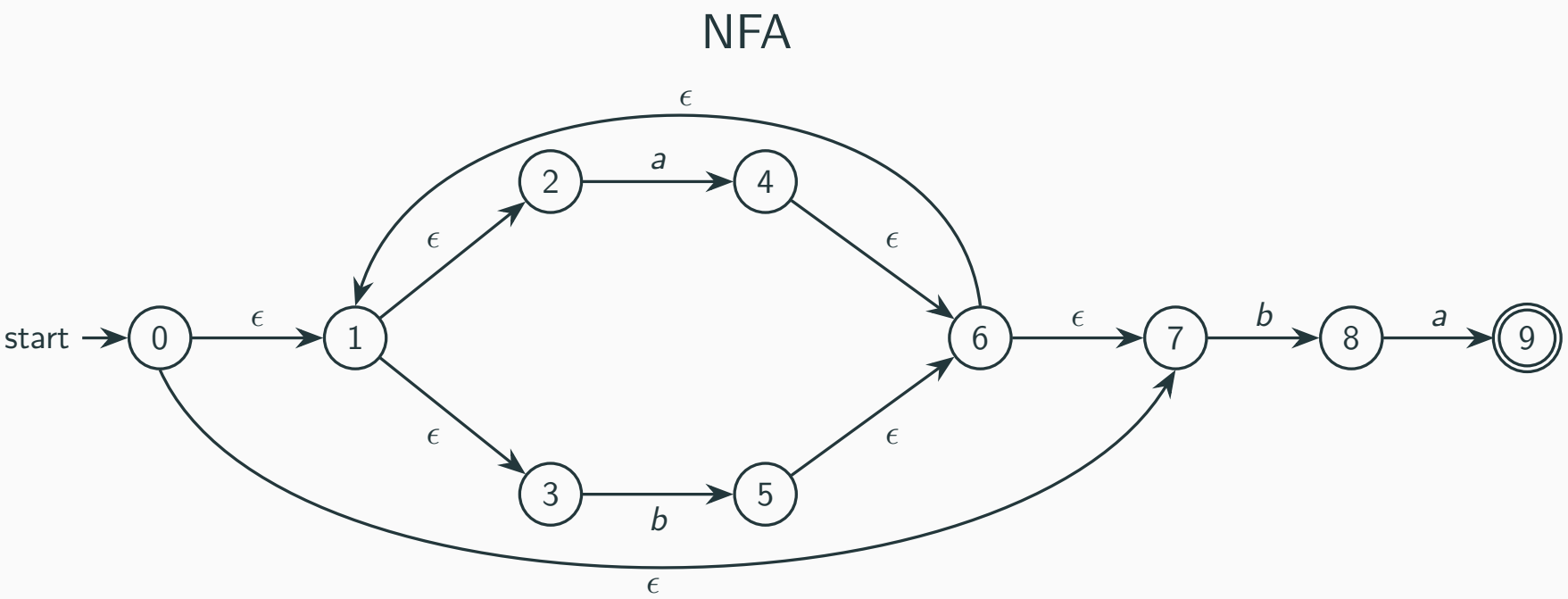


DFA

Passando dallo stato $A = \{q_0\}$ allo stato $B = \{q_0, q_1\}$, ho seguito tutte e due le transizioni b: $\mathit{mossa}(A, b) = \bigcup_{q \in A} \delta(q, b) = \delta(q_0, b) = B$.
Similmente, passando dallo stato $B = \{q_0, q_1\}$ a $C = \{q_0, q_2\}$, ho seguito sia la transizione $q_0 \xrightarrow{a} q_0$, sia la transizione $q_1 \xrightarrow{a} q_2$:

$\mathit{mossa}(B, a) = \bigcup_{q \in B} \delta(q, a) = \delta(q_0, a) \cup \delta(q_1, a) = \{q_0, q_2\} = C$.

Secondo esempio completo



- Lo stato iniziale del DFA è dato da 0, più tutti gli stati che posso raggiungere da 0 con mosse ϵ:

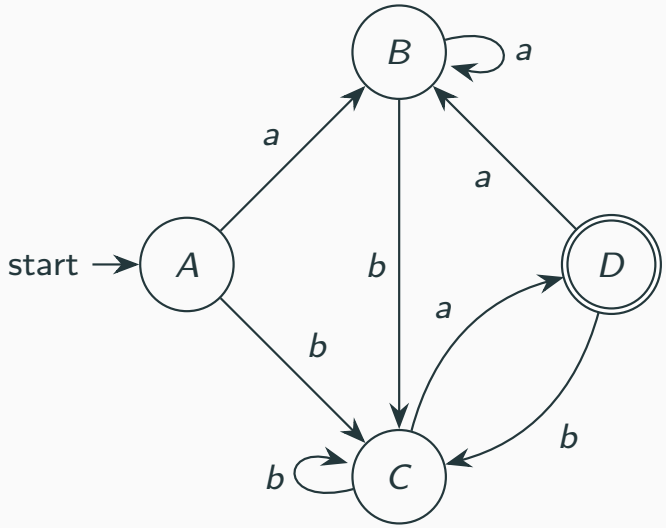
  $A = \epsilon\text{-}closure(\{0\}) = \{0, 1, 2, 3, 7\}$.

- Per calcolare lo stato che raggiungo da A leggendo a, devo vedere quali stati in A possono fare a, e poi fare la ϵ-closure di quelli raggiunti con quelle mosse etichettate a.

  $\Delta(A, a) = \epsilon\text{-}closure(mossa(A, a)) = \epsilon\text{-}closure(\bigcup_{q \in A} \delta(q, a))$
  $= \epsilon\text{-}closure(\{4\}) = \{4, 6, 7, 1, 2, 3\} = B$.

Secondo esempio completo (2)

- $\Delta(A, b) = \epsilon\text{-}closure(mossa(A, b)) = \epsilon\text{-}closure(\bigcup_{q \in A} \delta(q, b))$
  $= \epsilon\text{-}closure(\{5, 8\}) = \{\underline{5}, \underline{8}, 1, 2, 3, 6, 7\} = C.$
- $\Delta(B, a) = \epsilon\text{-}closure(\{4\}) = B.$
- $\Delta(B, b) = \epsilon\text{-}closure(\{5, 8\}) = C.$
- $\Delta(C, a) = \epsilon\text{-}closure(\{4, 9\}) = \{\underline{4}, \underline{9}, 1, 2, 3, 6, 7\} = D$ (9 è finale nell'NFA, quindi D è finale per il DFA).
- $\Delta(C, b) = \epsilon\text{-}closure(\{5, 8\}) = C.$
- $\Delta(D, a) = \epsilon\text{-}closure(\{4\}) = B.$
- $\Delta(D, b) = \epsilon\text{-}closure(\{5, 8\}) = C.$



DFA

ϵ-closure, mosse e funzione di transizione Δ del DFA

Sia q uno stato di un NFA, la *ϵ-closure* di q è l'insieme di stati raggiungibili da q solo con mosse ϵ, definita come il minimo insieme che soddisfa l'assioma e la regola qui sotto:

$$\frac{}{\{q\} \subseteq \epsilon\text{-}closure(q)} \qquad \frac{\{p\} \subseteq \epsilon\text{-}closure(q)}{\delta(p, \epsilon) \subseteq \epsilon\text{-}closure(q)}$$

Sia P un insieme di stati di un NFA.

$$\epsilon\text{-}closure(P) = \bigcup_{p \in P} \epsilon\text{-}closure(p)$$

$$mossa : \mathcal{P}(Q) \times \Sigma \to \mathcal{P}(Q)$$

$$mossa(P, b) = \bigcup_{p \in P} \delta(p, b)$$

$$\Delta(P, b) = \epsilon\text{-}closure(mossa(P, b))$$

Δ è la funzione di transizione del DFA.

Algoritmo per calcolare la ϵ-closure di P

Algoritmo: ϵ-closure(P)

$T \leftarrow P$
ϵ-closure(P) $\leftarrow P$
while $T \neq \emptyset$ **do**
    scegli un $r \in T$
    $T \leftarrow T \setminus \{r\}$
    for all $s \in \delta(r, \epsilon)$ **do**
        if $s \notin \epsilon$-closure(P) **then**
            aggiungi s a ϵ-closure(P)
            aggiungi s a T
        end if
    end for
end while

Osservazione: usando ϵ-closure, si può definire il linguaggio riconosciuto da un NFA in modo elegante usando la funzione $\hat{\delta} : Q \times \Sigma^* \rightarrow \mathcal{P}(Q)$

$$\hat{\delta}(q, \epsilon) = \epsilon\text{-}closure(q)$$
$$\hat{\delta}(q, xa) = \epsilon\text{-}closure(mossa(\hat{\delta}(q, x), a)) = \epsilon\text{-}closure(\cup_{p \in \hat{\delta}(q,x)} \delta(p, a))$$

Nota che $\boxed{w \in L[N] \text{ se e solo se } \exists p \in F \text{ tale che } p \in \hat{\delta}(q_0, w)}$

Subset construction

Dato un NFA $N = (\Sigma, Q, \delta, q_0, F)$:

Costruzione del DFA

$S \leftarrow \epsilon$-closure(q_0) ▷ S stato iniziale del DFA
$T \leftarrow \{S\}$ ▷ T insieme degli stati del DFA
▷ S inizialmente non è marcato

while esiste un $P \in T$ non marcato **do**
    marca P
    for all $a \in \Sigma$ **do**
        $R \leftarrow \epsilon$-closure$(mossa(P, a))$
        if $R \notin T$ **then**
            aggiungi R a T ▷ R non marcato
        end if
        $\Delta(P, a) \leftarrow R$
    end for
end while

Il DFA ottenuto con questo algoritmo è

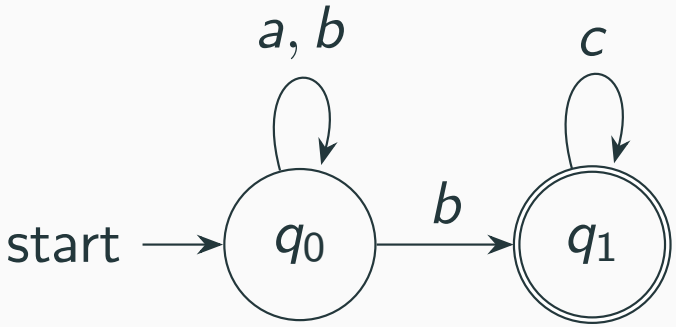
$$M_N = (\Sigma, T, \Delta, \epsilon\text{-}closure(q_0), \mathcal{F})$$

dove $R \in \mathcal{F}$ se e solo se $\exists q \in R$ con $q \in F$.

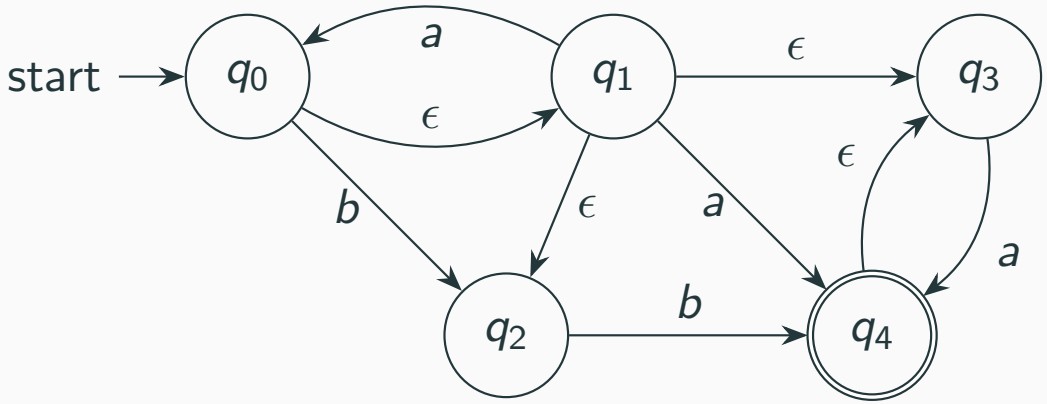
Caso pessimo

- Nel caso pessimo l'insieme degli stati T del DFA sarà equipotente all'insieme delle parti di Q, $T = \mathcal{P}(Q)$, cioè il DFA M_N costruito a partire dal NFA N ha un numero di stati pari a 2^n (esponenziale), se $n = |Q|$.
- Di solito T è molto più piccolo di $\mathcal{P}(Q)$ (vedi esempio completo su slide 36-37).

Esercizio: Prova a costruire il DFA (che avrà 2^2 stati) equivalente a questo NFA (con 2 stati).



Esempio

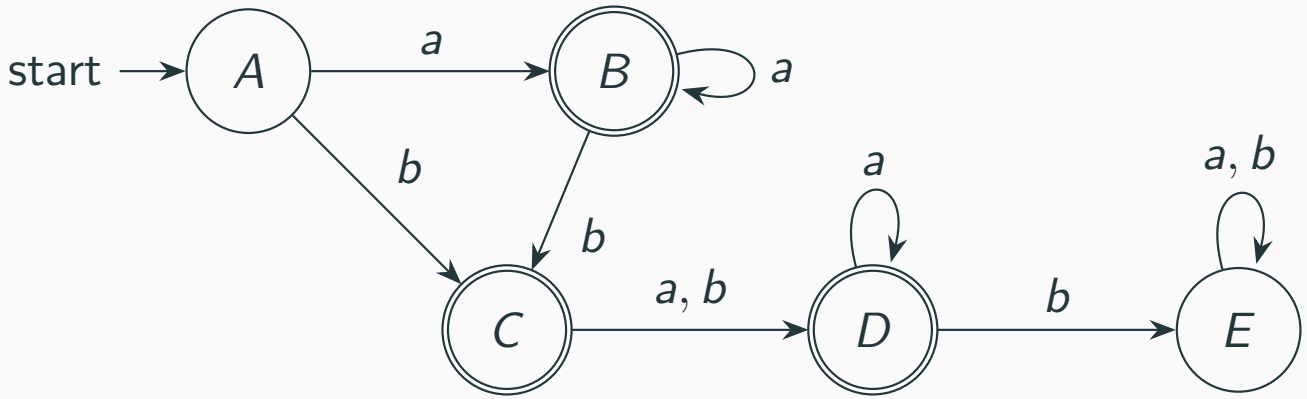


NFA N

- $A = \epsilon\text{-}closure(q_0) = \{q_0, q_1, q_2, q_3\}$
- $\Delta(A, a) = \epsilon\text{-}closure(mossa(A, a)) = \epsilon\text{-}closure(\{q_0, q_4\}) = \{q_0, q_1, q_2, q_3, q_4\} = B$
- $\Delta(A, b) = \epsilon\text{-}closure(mossa(A, b)) = \epsilon\text{-}closure(\{q_2, q_4\}) = \{q_2, q_3, q_4\} = C$
- $\Delta(B, a) = \epsilon\text{-}closure(mossa(B, a)) = \epsilon\text{-}closure(\{q_0, q_4\}) = B$
- $\Delta(B, b) = \epsilon\text{-}closure(mossa(B, b)) = \epsilon\text{-}closure(\{q_2, q_4\}) = C$
- $\Delta(C, a) = \epsilon\text{-}closure(mossa(C, a)) = \epsilon\text{-}closure(\{q_4\}) = \{q_3, q_4\} = D$
- $\Delta(C, b) = \epsilon\text{-}closure(mossa(C, b)) = \epsilon\text{-}closure(\{q_4\}) = D$

Esempio (2)

DFA M_N



- $\Delta(D, a) = \epsilon\text{-}closure(mossa(D, a)) = \epsilon\text{-}closure(\{q_4\}) = D$
- $\Delta(D, b) = \epsilon\text{-}closure(mossa(D, b)) = \epsilon\text{-}closure(\emptyset) = \emptyset = E$
- $\Delta(E, a) = \epsilon\text{-}closure(mossa(E, a)) = \epsilon\text{-}closure(\emptyset) = E$
- $\Delta(E, b) = \epsilon\text{-}closure(mossa(E, b)) = \epsilon\text{-}closure(\emptyset) = E$

Equivalenza NFA-DFA

Teorema
Sia $N = (\Sigma, Q, \delta, q_0, F)$ un NFA e sia M_N l'automa ottenuto con la costruzione per sottoinsiemi. Allora M_N è un DFA e si ha che

$$L[N] = L[M_N]$$

Corollario: la classe dei linguaggi riconosciuti dagli NFA coincide con la classe dei linguaggi riconosciuti dai DFA.

Dimostrazione del teorema

Sia $N = (\Sigma, Q, \delta, q_0, F)$ un NFA e sia $M_N = (\Sigma, T, \Delta, \epsilon\text{-}closure(q_0), \mathcal{F})$ l'automa ottenuto con l'algoritmo di cui sopra.

1. M_N è **deterministico**! Infatti, $\Delta(A, a)$ è definita per ogni coppia (A, a) con $A \in T$ e $a \in \Sigma$ in modo univoco, ed il risultato di $\Delta(A, a)$ è un elemento di T.
2. Quindi dobbiamo "solo" dimostrare che $L[N] = L[M_N]$
   - 2.1 Osservazione: per un DFA, $\epsilon\text{-}closure(R) = R$ con $R \in T$, poiché non ci sono mosse ϵ.
   - 2.2 Notazione: chiamiamo $i_M = \epsilon\text{-}closure(q_0)$ lo stato iniziale di M_N.

Vogliamo dimostrare che $\forall w \in \Sigma^*$

$$\hat{\delta}(q_0, w) = \hat{\Delta}(i_M, w)$$

per induzione sulla lunghezza di w.
(Per la definizione di $\hat{\delta}$ vedi slide 39)

Dimostrazione: caso base

Nel caso base $|w| = 0$, cioè $w = \epsilon$. Allora

- $\hat{\delta}(q_0, \epsilon) = \epsilon\text{-}closure(q_0)$
- $\hat{\Delta}(i_M, \epsilon) = \epsilon\text{-}closure(i_M) = i_M = \epsilon\text{-}closure(q_0)$

E quindi la tesi è soddisfatta: $\hat{\delta}(q_0, \epsilon) = \hat{\Delta}(i_M, \epsilon)$.

Dimostrazione: passo induttivo

Sia $w = xa$ con $a \in \Sigma$ e $x \in \Sigma^*$.

Per ipotesi induttiva, sappiamo che

$$\hat{\delta}(q_0, x) = \hat{\Delta}(i_M, x) = \{p_1, ..., p_k\}$$

Per definizione di $\hat{\delta}$, sappiamo che

$$\hat{\delta}(q_0, xa) = \epsilon\text{-}closure(mossa(\hat{\delta}(q_0, x), a)) = \epsilon\text{-}closure(\bigcup_{i=1}^{k} \delta(p_i, a))$$

e similmente

$$\hat{\Delta}(i_M, xa) = \Delta(\{p_1, ..., p_k\}, a)$$

In base all'algoritmo, la definizione di Δ ci dice che

$$\begin{aligned}\Delta(\{p_1, ..., p_k\}, a) &= \epsilon\text{-}closure(mossa(\{p_1, ..., p_k\}, a)) = \\ &= \epsilon\text{-}closure(\bigcup_{i=1}^{k} \delta(p_i, a)) \\ &= \hat{\delta}(q_0, xa)\end{aligned}$$

Dimostrazione: conclusione

Infine, abbiamo che $\forall w \in \Sigma^*$

$$\begin{aligned} w \in L[N] &\iff \exists p \in \hat{\delta}(q_0, w) \text{ con } p \in F \\ &\iff \exists p \in \hat{\Delta}(i_M, w) \text{ con } p \in F \\ &\iff \hat{\Delta}(i_M, w) \in \mathcal{F} \\ &\iff w \in L[M_N] \end{aligned}$$

Quindi $L[N] = L[M_N]$, c.v.d.